

Task2 - GAN-based Anomaly Detector

- The main goal of this task is to implement a GAN-based anomaly detection system for ICS network traffic:
 - The objective is to classify network traffic samples as:
 - Normal traffic
 - Anomalous traffic (attacks)
 - We should work on raw packet bytes directly.
 - Input representation:
 - The GAN must operate on raw network packets, represented as a matrix of size:
 - $\text{size} = m * n$
 - m = number of packets in the dataset
 - $n = M$ = number of bytes extracted per packet.
 - M should be a parameter that is read from the command line.
 - Bytes are taken directly from packets.
 - Data can be:
 - Full packets
 - Reverse-engineered packet parts (will be used later in Task3).
 - GAN Architecture overview:
 - It must be a Generative Adversarial Network (GAN) consisting of:
 - Discriminator (D)
 - Acts as anomaly detector / classifier.
 - Generator (G)
 - Creates fake packet data that should resemble real network packets.
 - These two networks are trained adversarially:
 - The D learns to distinguish real vs fake.
 - The G learns to fool the D.
 - D Design:
 - The discriminator must be a deep CNN with the following structure:
 - Convolutional Part:
 - 9 convolutional layers.
 - ReLU activation in all layers
 - Dropout after every convolutional layer except:
 - The first layer
 - The last layer
 - Fully Connected Part:
 - output of the final convolutional layer is flattened
 - Followed by 4 fully connected layers.
 - ReLU activation in all layers except the last layer.
 - Output:
 - A single scalar value in range $[0,1)$.
 - Represents how *real* the input sample is
 - Used for:
 - GAN training.
 - Anomaly detection (option 1).
 - G Design:
 - The generator must function as reverse CNN that transforms noise into packet-like data.
 - Input:
 - Random noise vector
 - Noise distribution is our choice.
 - Fully Connected Layers:

- 4 fully connected layers
 - LeakyReLU activation.
 - Convolutional Layers:
 - 8 convolutional layers with LeakyReLU
 - First 4 convolutional layers must be preceded by upsampling
 - Output:
 - Generated packets bytes matching the same shape as real input.
 - Output should resemble real packet byte distributions.
- Custom Generator Loss
 - We should not use a standard GAN loss alone
 - Instead:
 - Implement a custom loss function for the generator
 - The loss is computed using the input to the first fully connected layer of the discriminator.
 - What this means:
 - We extract intermediate feature representations from the D.
 - Compute the difference between:
 - Features of real packets.
 - Features of generated packets.
 - Purpose:
 - Forces the generator to match semantic packet features, not just byte-level similarity
 - Improves anomaly detection robustness.
 - This is called feature-matching loss.
- Anomaly Detection modes (Runtime choice):
 - After training, our system should support two anomaly detection strategies, selectable via command line:
 - Mode1:
 - Use D directly
 - Low realism score -> Anomalous
 - Fast and simple:
 - Mode2:
 - Use generator behavior to detect anomalies
 - Example:
 - Reconstruction error
 - Feature mismatch
 - More computationally expensive, but sometimes more robust.
- Hyperparameter Tuning:
 - We must implement hyperparameter tuning for GAN
 - Example on tunable parameters:
 - Learning rate
 - Number of epochs
 - Kernel size
 - Number of filters
 - Dropout rates
 - This can be done via:
 - Grid search
 - Manual tuning
 - Automated tuning
 - The main goal is to show experimental rigor.
- How task2 is used in Task3:
 - In Task3, our GAN will be evaluated in Scenario 1 with:
 - Full packets or reverse-engineered packet chunks.
 - Physical reading sizes $p = \{5, 10, 15\}$.
 - Raw bytes only (no autoencoder features).
 - We must report:
 - Precision

- Recall
 - Comparative plots
 - Clear experimental conclusions.
 - The GAN is evaluated twice:
 - Using D
 - Using G
- Now a plan for how to proceed with the implementation:
 - Phase 0:
 - Design Decisions
 - We should decide now:
 - Which framework to use (PyTorch or TensorFlow)
 - I will use PyTorch
 - Input shape conventions:
 - (batch_size, 1,m,n) or (batch_size, m, n)
 - I will use (batch_size, 1, m, n)
 - This is the standard CNN format:
 - number of samples => batch_size
 - number of channels => 1 (bytes are single channel)
 - number of packets -> m
 - number of bytes per packet -> n
 - and the CNN usually expects data in this format because it was originally designed for images:
 - So now we will have image similarity where:
 - height = m (number of packets)
 - width = n (number of bytes per packet)
 - Packet extraction rule:
 - First M bytes per packet
 - Noise distribution:
 - I will use normal distribution (standard Gaussian)
- Phase 1:
 - Data pipeline
 - 1. Command-line interface
 - I will implement arguments as follows:
 - --n : number of bytes per packet (M)
 - --mode : anomaly detection mode (D or G)
 - 2. Packets preprocessing:
 - We should have a function that load the dataset in form of raw packets, and return it in shape of (m, n)
 - And it should truncate/pad each packet to M bytes
 - It is already implemented before, but we will just need to adapt it here.
 - 3. Dataset splits
 - Prepare the dataset into training and testing sets
 - Train set contains normal packets only
 - Validation set contains normal and anomalous packets
 - Test set contains both normal and anomalous packets
 - This is important for tuning and evaluation
- Phase 2: Discriminator implementation
 - Architecture:
 - 9 convolutional layers (ReLU)
 - Dropout after every conv layer except first and last
 - 4 fully connected layers (ReLU)
 - Final sigmoid output
 - Feature extractor separation:
 - We should refactor Discriminator into:
 - D_features(X) -> returns feature vector
 - D_classifier(features) -> returns probability [0,1]
 - This is mandatory for:
 - Custom generator loss
 - Generator-based anomaly detection

- Sanity Checks:
 - Before GAN training, we must ensure that:
 - Forward pass works
 - Feature tensor shape is stable
 - Output is in $[0,1]$
- Phase 3: Generator implementation
 - Noise input
 - We should define input as $m \times n$ also sampled from normal distribution
 - $z = N(0,1)$
 - Architecture:
 - 4 fully connected layers (LeakyReLU)
 - 8 convolutional layers (LeakyReLU)
 - Upsampling before first 4 conv layers
 - Final output layer to match $(1,m,n) \rightarrow$ real packet shape
 - Output constraints:
 - Output range should match normalized packet range
 - No activation mismatches (e.g. sigmoid vs tanh confusion)
- Phase 4: Custom Generator Loss
 - Feature extraction:
 - For each batch:
 - $f_real = D_features(X_real)$
 - $f_fake = D_features(G(z))$
 - Loss computation:
 - we can use L1 or L2 distance between mean feature vectors:
 - $L_G = \| \text{mean}(f_real) - \text{mean}(f_fake) \|$
 - Training Logic:
 - Freeze D weights during G update
 - Update G only using custom loss
- Phase 5: GAN Training Loop
 - D step:
 - Train to:
 - Classify real samples as 1
 - Classify generated samples as 0
 - G step:
 - Train to:
 - Minimize feature mismatch
 - Fool discriminator indirectly
 - Logging:
 - D loss
 - G feature loss
 - Optional discriminator accuracy
- Phase 6: Anomaly Detection Modes
 - Mode 1 (Discriminator-based):
 - For each test sample:
 - Compute $D(X)$
 - If $D(X) < \text{threshold} \rightarrow$ Anomalous
 - Mode 2 (Generator-based):
 - For each test sample:
 - Find closest $G(z)$ to X
 - Compute reconstruction error or feature mismatch
 - If error $> \text{threshold} \rightarrow$ Anomalous
- Phase 7: Hyperparameter Tuning
 - Tune:
 - Learning rate
 - Number of epochs
 - Kernel size
 - Dropout rates
 - Use grid search
- Phase 8: prepare for Task3 integration
 - We should consider during implementation:

- Accept full packets or reverse-engineered chunks
- Works for $p = \{5, 10, 15\}$
- Output precision/recall metrics